End-to-End Data Management Pipeline for a Recommendation System

Syed Amaan Hasan | ID 2025AG05941 | Data Management for Machine Learning (DM4ML), Assignment 2

1. Introduction and problem

This project implements a reproducible recommendation pipeline for an online movie platform using MovieLens 100K. The pipeline covers ingestion, validation, preparation, transformation, feature management, model training, evaluation, and orchestration. The objective is a clean, modular, auditable data-management workflow rather than a leaderboard score.

2. Architecture and design decisions

The pipeline is deliberately linear and idempotent: each stage writes Parquet/CSV evidence that the next stage reads. Invalid rows are quarantined instead of silently dropped. The train/test split is strictly temporal, and every feature is built from the training window with an explicit as-of date to prevent leakage.

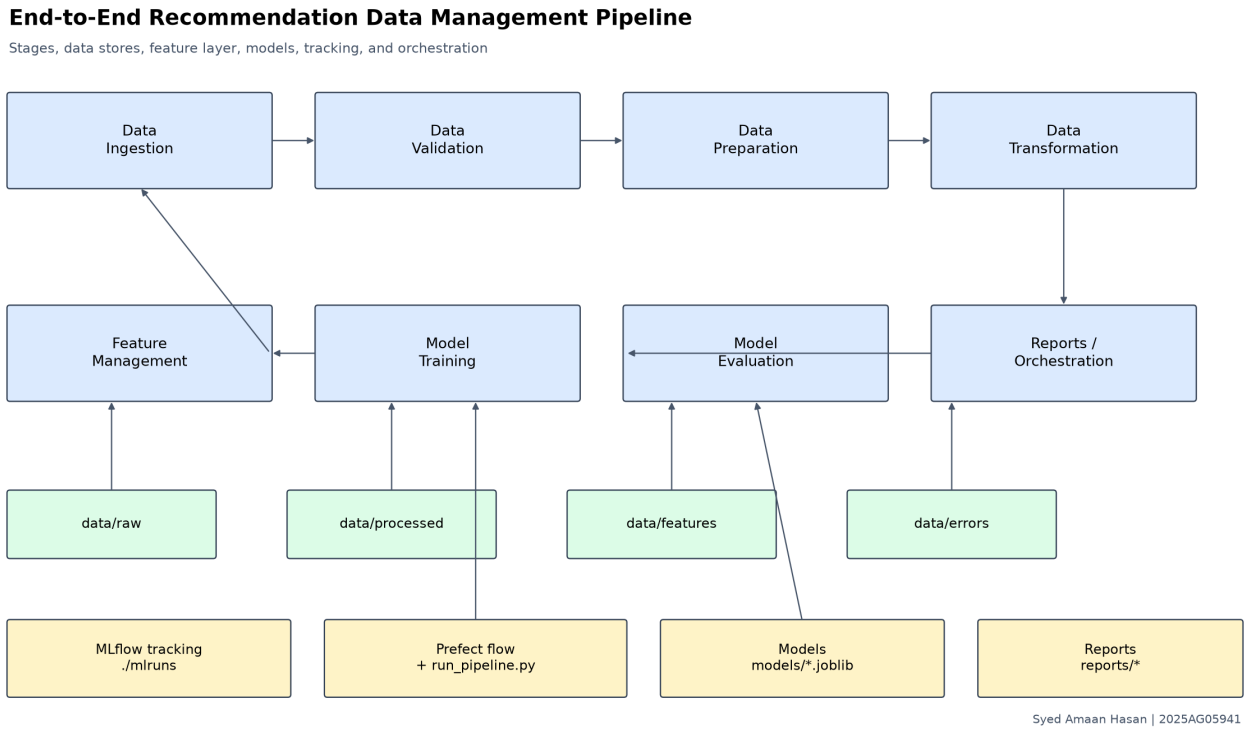


Figure 1: Pipeline architecture and data stores.

3. Tools used and why

Python, pandas, NumPy, SciPy, and PyArrow provide the data layer; Pandera supplies schema validation; scikit-learn TruncatedSVD supplies the personalized latent-factor model; MLflow records parameters, metrics, and artifacts; Prefect plus a plain CLI runner provide orchestration; matplotlib and ReportLab produce the evidence figures and this PDF.

4. Data validation findings

Dataset	Rows in	Valid	Rejected	Pandera
users	945	943	2	PASS
items	1684	1682	2	PASS
interactions	100001	99991	10	PASS

The validation stage checked missing values, duplicates, dtypes, rating ranges, timestamp sanity, and referential integrity. The demo run injected faults: yes. Every injected fault was detected and written to data/errors/ with a failed_rules column; the clean rows continued through the pipeline.

5. Preparation, transformation, and split strategy

The split method is **per_user_last_fraction** (test fraction 0.2). Per-user training cutoffs range from 1997-09-20 20:17:44+00:00 to 1998-04-22 23:10:38+00:00. Train has 79614 interactions and test has 20377; the no-test-before-cutoff guard is True. Cold-start test users: 0; cold-start test items: 71. Users are warm by construction in per-user mode; unseen items are not recommendable and therefore count as misses.

6. Feature management

The FeatureStore writes user, item, and interaction features to data/features/ and serves the same tables for prediction. Features use an explicit as-of date of 1998-04-22 23:10:38+00:00, derived from the training window. The generated feature catalog is in reports/feature_catalog.md and .csv; it is built from the same registry used by the implementation.

7. Models

The baseline ranks unseen items by training popularity. The personalized model learns latent ranking factors with TruncatedSVD on the sparse user-item rating matrix, and uses a separate bias-aware residual SVD (global mean plus user/item offsets) for explicit rating prediction. Both implement the same fit/recommend interface, and both were tracked in separate MLflow runs.

8. Results

model	precision@10	recall@10	ndcg@10	coverage@10	diversity@10	rmse	mae
popularity	0.0754	0.0749	0.0943	0.0447	0.7478	1.0735	0.8587
matrix_factorization	0.1320	0.1543	0.1763	0.2862	0.7128	1.0082	0.7915

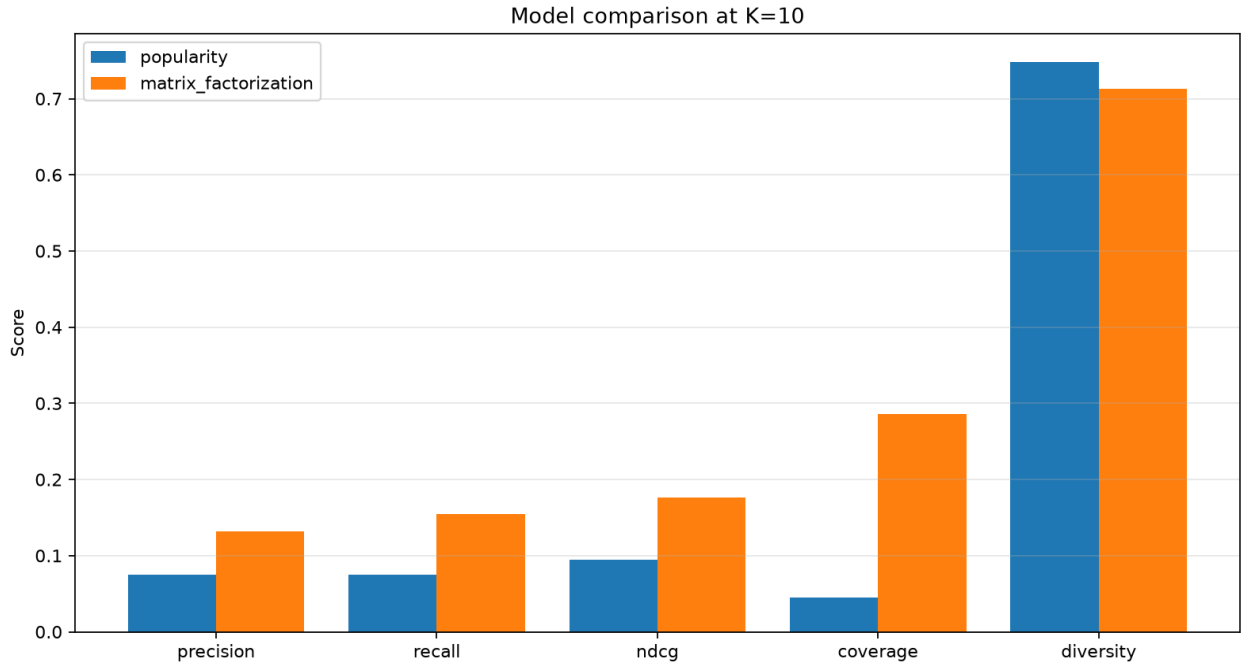


Figure 2: Model comparison at K=10.

The personalized model improves every ranking metric over the popularity baseline, including a much larger share of the catalog covered. Its slightly lower diversity reflects more concentrated, user-specific lists. The popularity baseline remains a strong, cheap benchmark, but the latent-factor model is the better ranker here.

9. Orchestration and reproducibility

The Prefect flow in dags/recommendation_flow.py has one task per stage with retries and explicit dependencies; run_pipeline.py executes the same stage functions sequentially for environments without a Prefect server. Seeds and the split rule are config-driven, so a clean rerun produces the same metrics and tables. reports/run_log.txt and reports/figures/orchestration_success.png capture the actual successful run. The orchestration image is generated from real stage timings and is explicitly labelled as not a Prefect UI screenshot.

10. Limitations and future work

MovieLens 100K is small and explicit-feedback only. The SVD model does not use content features or sequence context, and the per-user temporal split still creates cold-start items. Future work could add implicit-feedback ALS, content-based hybrid features, stronger bias regularization, and a feature store with online serving. The pipeline intentionally keeps the feature layer batch-oriented for reproducibility.